

SIMATS ENGINEERING

ASSESSMENT TOOL 1-Constraint Based Problem Solving

COURSE NAME: R PROGRAMMING

COURSE CODE: ITA04

COURSE OUTCOME COVERED

CO: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively.
(Bloom Level -BL2- BL4)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks: 100

1.Student Marks Analysis (Vectors) – BL2

Problem Statement:

A teacher has stored the marks of 10 students in a vector.

Constraints:

- Do not use built-in functions like `sort()`.
- Use only vector indexing and arithmetic operators.
- Display the highest, lowest, and average marks.

```
monthlySales <- c(
12000,14000,13000,15000,
14500,16000,17000,16500,
15500,18000,17500,19000)
```

```
quarterlySales <- matrix(c(
39000,45500,49000,54500,
41000,47000,50000,56000),
nrow=2,byrow=TRUE)
```

```
regionalSales <- array(c(
10000,12000,
11000,13000,
9000,14000,
10500,12500,
```

```

11500,13500,
9500,14500,
9800,12800,
11800,13800),
dim=c(2,2,2))

categories <- factor(c(
"Electronics",
"Grocery",
"Stationery",
"Electronics"))

print(class(monthlySales))
print(class(quarterlySales))
print(class(regionalSales))
print(class(categories))

annualSales <- sum(monthlySales)

cat("Annual Sales =",annualSales,"\n")

regionTotal <- c()

for(i in 1:2)
{
  total <- 0

  for(j in 1:2)
  {
    for(k in 1:2)
    {
      total <- total + regionalSales[i,j,k]
    }
  }

  regionTotal[i] <- total
}

print(regionTotal)

bestRegion <- which.max(regionTotal)

cat("Best Performing Region =",bestRegion,"\n")

quarterTotal <- rowSums(quarterlySales)

print(quarterTotal)

```

```
<
> cat("Highest Marks =", highest, "\n")
Highest Marks = 95
> cat("Lowest Marks =", lowest, "\n")
Lowest Marks = 68
> cat("Average Marks =", average, "\n")
Average Marks = 82.5
> |
```

2. Employee Salary Matrix – BL3

Problem Statement:

A company stores employee salaries of four departments in a 4×3 **matrix**, where rows represent departments and columns represent Basic Pay, HRA, and DA.

Constraints:

- Do not create separate vectors.
- Calculate total salary department-wise using matrix operations only.
- Display the department with the maximum total salary.

```
salary <- matrix(c(
  30000,5000,4000,
  35000,6000,4500,
  28000,4500,3500,
  40000,7000,5000),
  nrow=4,byrow=TRUE)

print(salary)

totalSalary <- salary[,1] + salary[,2] + salary[,3]

print(totalSalary)

maxDept <- 1

for(i in 2:4)
{
  if(totalSalary[i] > totalSalary[maxDept])
    maxDept <- i
}

cat("Department with Maximum Salary =", maxDept, "\n")
```

```

> print(salary)
      [,1] [,2] [,3]
[1,] 30000 5000 4000
[2,] 35000 6000 4500
[3,] 28000 4500 3500
[4,] 40000 7000 5000
>
> totalSalary <- salary[,1] + salary[,2] + salary[,3]
>
> print(totalSalary)
[1] 39000 45500 36000 52000
>
> maxDept <- 1
>
> for(i in 2:4)
+ {
+   if(totalSalary[i] > totalSalary[maxDept])
+     maxDept <- i
+ }
>
> cat("Department with Maximum Salary =", maxDept, "\n")
Department with Maximum Salary = 4

```

3. Rainfall Data (Arrays) – BL3

Problem Statement:

A meteorological department records rainfall for **3 cities**, **4 seasons**, and **2 years** in a three-dimensional array.

Constraints:

- Use only array indexing.
- Find the city with the highest annual rainfall.
- Do not convert the array into a matrix or data frame.

```

rain <- array(c(
  100,120,110,130,
  90,100,95,105,
  80,85,90,95,

  110,125,115,135,
  95,105,100,110,
  85,90,95,100),
  dim=c(3,4,2))

```

```
print(rain)
```

```
highest <- 0
```

```

city <- 1

for(i in 1:3)
{
  total <- 0

  for(j in 1:4)
  {
    for(k in 1:2)
    {
      total <- total + rain[i,j,k]
    }
  }

  cat("City",i,"Rainfall =",total,"\n")

  if(total > highest)
  {
    highest <- total
    city <- i
  }
}

cat("City with Highest Rainfall =",city,"\n")
+ }
+ }
City 1 Rainfall = 845
City 2 Rainfall = 830
City 3 Rainfall = 790
>
> cat("City with Highest Rainfall =",city,"\n")
City with Highest Rainfall = 1
>

```

4. Hospital Patient Records (Lists) – BL4

Problem Statement:

A hospital stores patient details (ID, Name, Age, Blood Group, Bill Amount) in a list.

Constraints:

- Store different data types in a single list.
- Retrieve only patients aged above 60.
- Calculate the average hospital bill without converting the list into another data structure.

```

patient <- list(
ID=c(101,102,103,104),
Name=c("Ravi","Anu","Kumar","Sita"),
Age=c(65,45,70,62),

```

```

BloodGroup=c("A+","B+","O+","AB+"),
Bill=c(25000,18000,32000,27000)
)

print(patient)

cat("Patients above 60 Years\n")

for(i in 1:length(patient$Age))
{
  if(patient$Age[i] > 60)
  {
    cat(patient$ID[i],
        patient$Name[i],
        patient$Age[i],"\n")
  }
}

total <- 0

for(i in 1:length(patient$Bill))
{
  total <- total + patient$Bill[i]
}

average <- total/length(patient$Bill)

cat("Average Bill =",average,"\n")
+ }
+ }
101 Ravi 65
103 Kumar 70
104 Sita 62
>
> total <- 0
>
> for(i in 1:length(patient$Bill))
+ {
+   total <- total + patient$Bill[i]
+ }
>
> average <- total/length(patient$Bill)
>
> cat("Average Bill =",average,"\n")
Average Bill = 25500
> |

```

5. Product Inventory (Factors) – BL3

Problem Statement:

A supermarket stores product categories (Electronics, Grocery, Stationery) using factors.

Constraints:

- Count the number of products in each category.
- Do not use `table()`.
- Display categories in alphabetical order.

```
products <- factor(c(
  "Electronics",
  "Grocery",
  "Stationery",
  "Electronics",
  "Grocery",
  "Grocery",
  "Stationery",
  "Electronics"))
```

```
levels(products) <- sort(levels(products))
```

```
print(products)
```

```
countE <- 0
```

```
countG <- 0
```

```
countS <- 0
```

```
for(i in 1:length(products))
{
  if(products[i]=="Electronics")
    countE <- countE+1

  if(products[i]=="Grocery")
    countG <- countG+1

  if(products[i]=="Stationery")
    countS <- countS+1
}
```

```
cat("Electronics =",countE,"\n")
```

```
cat("Grocery =",countG,"\n")
```

```
cat("Stationery =",countS,"\n")
```

```

> counts <- 0
> countsS <- 0
>
> for(i in 1:length(products))
+ {
+   if(products[i]=="Electronics")
+     countE <- countE+1
+
+   if(products[i]=="Grocery")
+     countG <- countG+1
+
+   if(products[i]=="Stationery")
+     countsS <- countsS+1
+ }
>
> cat("Electronics =",countE,"\n")
Electronics = 3
> cat("Grocery =",countG,"\n")
Grocery = 3
> cat("Stationery =",countsS,"\n")
Stationery = 2
> |

```

6. Cricket Performance Analysis (Data Frames) – BL4

Problem Statement:

A cricket academy maintains player records containing Name, Matches, Runs, Wickets, and Strike Rate.

Constraints:

- Identify players with Strike Rate greater than 120.
- Compute the average runs scored.
- Do not use SQL queries or external packages.

```

players <- data.frame(
  Name=c("Rohit","Virat","Gill","Hardik","Surya"),
  Matches=c(20,22,18,15,19),
  Runs=c(900,1100,780,500,820),
  Wickets=c(0,1,2,18,0),
  StrikeRate=c(135,128,118,130,145)
)

```



```

print(players)

cat("Players with Strike Rate >120\n")

for(i in 1:nrow(players))
{
  if(players$StrikeRate[i] > 120)
    print(players[i,])
}

total <- 0

for(i in 1:nrow(players))
{
  total <- total + players$Runs[i]
}

average <- total/nrow(players)

cat("Average Runs =",average,"\n")
+ }
  Name Matches Runs Wickets StrikeRate
1 Rohit      20  900      0      135
  Name Matches Runs Wickets StrikeRate
2 Virat      22 1100      1      128
  Name Matches Runs Wickets StrikeRate
4 Hardik     15  500     18      130
  Name Matches Runs Wickets StrikeRate
5 Surya      19  820      0      145
>
> total <- 0
>
> for(i in 1:nrow(players))
+ {
+   total <- total + players$Runs[i]
+ }
>
> average <- total/nrow(players)
>
> cat("Average Runs =",average,"\n")
Average Runs = 820

```

7. Bank Transaction Processing (Class Types) – BL3

Problem Statement:

A bank stores account numbers, customer names, balances, and account status.

Constraints:

- Identify the class of every column.
- Convert the account status into a factor.
- Ensure balances remain numeric after conversion.

```
bank <- data.frame(
  AccountNo=c(1001,1002,1003),
  CustomerName=c("Ravi","Anu","John"),
  Balance=c(25000,18000,32000),
  Status=c("Active","Inactive","Active")
)
```

```
print(bank)
```

```
print(class(bank$AccountNo))
print(class(bank$CustomerName))
print(class(bank$Balance))
print(class(bank$Status))
```

```
bank$Status <- factor(bank$Status)
```

```
cat("After Conversion\n")
```

```
print(class(bank$Status))
print(class(bank$Balance))
```

```
print(bank)
  AccountNo CustomerName Balance Status
1      1001         Ravi  25000  Active
2      1002         Anu  18000 Inactive
3      1003         John  32000  Active
```

```
>
> print(class(bank$AccountNo))
[1] "numeric"
> print(class(bank$CustomerName))
[1] "character"
> print(class(bank$Balance))
[1] "numeric"
> print(class(bank$Status))
[1] "character"
>
> bank$Status <- factor(bank$Status)
>
> cat("After Conversion\n")
After Conversion
>
> print(class(bank$Status))
[1] "factor"
> print(class(bank$Balance))
[1] "numeric"
```

8. University Attendance Management (Vectors and Logical Operations) – BL4

Problem Statement:

Attendance percentages of 15 students are stored in a vector.

Constraints:

- Students with attendance below 85% are not eligible for examinations.
- Display eligible and ineligible students separately.
- Calculate the percentage of eligible students.

```
attendance <- c(
90,78,88,82,95,
84,91,80,87,92,
75,86,89,81,93)
```

```
eligible <- c()
ineligible <- c()
```

```
count <- 0
```

```
for(i in 1:length(attendance))
{
  if(attendance[i] >= 85)
  {
    eligible <- c(eligible,i)
    count <- count+1
  }
  else
  {
    ineligible <- c(ineligible,i)
  }
}
```

```
cat("Eligible Students =",eligible,"\n")
cat("Ineligible Students =",ineligible,"\n")
```

```
percentage <- (count/length(attendance))*100
```

```
cat("Eligible Percentage =",percentage,"\n")
```

```

>
> cat("Eligible students =",eligible,"\n")
Eligible students = 1 3 5 7 9 10 12 13 15
> cat("Ineligible students =",ineligible,"\n")
Ineligible students = 2 4 6 8 11 14
>
> percentage <- (count/length(attendance))*100
>
> cat("Eligible Percentage =",percentage,"\n")
Eligible Percentage = 60

```

9. Electricity Consumption Analysis (Matrices) – BL4

Problem Statement:

Monthly electricity consumption of **5 buildings** over **12 months** is stored in a matrix.

Constraints:

- Calculate annual consumption for each building.
- Identify the building with the highest consumption.
- Use matrix operations only.

```

electricity <- matrix(c(
120,110,130,125,118,122,135,140,145,138,130,128,
100,105,110,115,108,112,118,120,125,122,119,121,
140,145,150,148,142,146,152,155,160,158,150,149,
90,95,100,98,96,102,105,108,110,109,104,106,
130,135,140,138,136,142,145,148,150,149,144,146),
nrow=5,byrow=TRUE)

print(electricity)

annual <- rowSums(electricity)

print(annual)

building <- which.max(annual)

cat("Building with Highest Consumption =",building,"\n")

```

```

>
> print(electricity)
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12]
[1,]  120  110  130  125  118  122  135  140  145  138  130  128
[2,]  100  105  110  115  108  112  118  120  125  122  119  121
[3,]  140  145  150  148  142  146  152  155  160  158  150  149
[4,]   90   95  100   98   96  102  105  108  110  109  104  106
[5,]  130  135  140  138  136  142  145  148  150  149  144  146
>
> annual <- rowSums(electricity)
>
> print(annual)
[1] 1541 1375 1795 1223 1703
>
> building <- which.max(annual)
>
> cat("Building with Highest Consumption =",building,"\n")
Building with Highest Consumption = 3

```

10. Sales Performance Dashboard (Vectors, Matrices, Arrays, and Class Types) – BL4

Problem Statement:

A retail company stores:

- Monthly sales in a vector,
- Quarterly sales in a matrix,
- Regional sales in a three-dimensional array,
- Product categories as factors.

Constraints:

- Compute total annual sales.
- Identify the best-performing region.
- Compare quarterly performance.
- Verify the class type of every object before performing calculations.
- Use only base R functions.

```

monthlySales <- c(
  12000,14000,13000,15000,
  14500,16000,17000,16500,
  15500,18000,17500,19000)

```

```

quarterlySales <- matrix(c(
  39000,45500,49000,54500,
  41000,47000,50000,56000),
  nrow=2,byrow=TRUE)

```

```

regionalSales <- array(c(
  10000,12000,

```

```
11000,13000,  
9000,14000,  
10500,12500,  
11500,13500,  
9500,14500,  
9800,12800,  
11800,13800),  
dim=c(2,2,2))
```

```
categories <- factor(c(  
"Electronics",  
"Grocery",  
"Stationery",  
"Electronics"))
```

```
print(class(monthlySales))  
print(class(quarterlySales))  
print(class(regionalSales))  
print(class(categories))
```

```
annualSales <- sum(monthlySales)
```

```
cat("Annual Sales =",annualSales,"\n")
```

```
regionTotal <- c()
```

```
for(i in 1:2)  
{  
  total <- 0  
  
  for(j in 1:2)  
  {  
    for(k in 1:2)  
    {  
      total <- total + regionalSales[i,j,k]  
    }  
  }  
  
  regionTotal[i] <- total  
}
```

```
print(regionTotal)
```

```
bestRegion <- which.max(regionTotal)
```

```
cat("Best Performing Region =",bestRegion,"\n")
```

```
quarterTotal <- rowSums(quarterlySales)
```

```
print(quarterTotal)
```

```
+ }  
>  
> print(regionTotal)  
[1] 40500 51500  
>  
> bestRegion <- which.max(regionTotal)  
>  
> cat("Best Performing Region =",bestRegion,"\n")  
Best Performing Region = 2  
>  
> quarterTotal <- rowSums(quarterlySales)  
>  
> print(quarterTotal)  
[1] 188000 194000  
> |
```

Code of Conduct:

*I **TANUJA P (192321052)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Tanuja P

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation

Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient